DashVMC

Real-Time Discrete World Model Control in Geometry Dash

Florent Tariolle

INSA Rouen Normandy

World-model agents are usually evaluated in simulators that can wait for the policy; live games impose the opposite constraint, requiring capture, prediction, and action before the next frame. We present *DashVMC*, a discrete Vision–Model–Controller system that plays Geometry Dash from screen pixels. An FSQ tokenizer maps 64×64 Sobel frames to 8×8 token grids, a block-causal transformer predicts action-conditioned transitions, and a lightweight actor–critic is initialized by behavioural cloning (BC) and subsequently optimized with PPO in latent rollouts. Training uses approximately 2 hours of gameplay captured at a nominal 30-FPS cadence, including only 20 minutes of expert segments; PPO requires no additional environment interaction. The selected PPO policy more than doubles the mean survival of an epoch-9 reconstructed BC comparator on each of the first two official levels, with a smaller gain on the harder third level; its PPO > reconstructed BC > no-op ordering also holds on a late Level-1 community copy and a held-out community layout within the supported mechanics. During a configured 60-FPS live probe on an RTX 2060 SUPER, the optimized capture-to-action loop averages 14.4 ms (p95 16.1 ms; maximum 17.0 ms); a separate 5,000-frame profile averages 16.3 ms. On the late Level-1 copy, the 60-FPS probe yields similar observed wall-clock survival to a 30-FPS diagnostic-path baseline. Beyond control, the same recurrent dynamics support interactive, open-ended visual level continuations that can remain plausible in recorded examples before autoregressive errors accumulate. Together, these results show that a compact discrete world-model controller can support generative rollouts and live 60-FPS reactive control on a consumer GPU, with the live policy ordering consistent with imagined PPO optimization transferring to deployment.

Project page: tariolle.github.io/dash-vmc
Code: github.com/Tariolle/dash-vmc
Correspondence: `florent.tariolle@insa-rouen.fr`

1 Introduction

World models learn the consequences of actions and can move policy improvement from costly environment interaction into imagination (Ha and Schmidhuber, 2018). Discrete approaches such as IRIS encode images into tokens and model their evolution with an autoregressive transformer (Micheli et al., 2023); DIAMOND instead shows that preserving fine visual detail with a diffusion model can improve control (Alonso et al., 2024). Most evaluations, however, run inside an emulator or a learned simulator whose clock effectively waits for inference. An agent controlling a live, non-paused application must also satisfy a wall-clock contract: screen capture, preprocessing, state estimation, policy inference, and action dispatch must finish before the action becomes stale.

Geometry Dash is a small but unforgiving test of this contract. The avatar moves continuously, the action space is binary, and a mistimed jump causes immediate death. Official levels are deterministic, but the agent observes only captured pixels and acts through keyboard events; it does not receive emulator state during control. This combination isolates two practical questions: can a compact visual world model support policy learning, and can the resulting perception-to-action path run within a consumer-GPU frame budget?

DashVMC follows the Vision–Model–Controller decomposition (Ha and Schmidhuber, 2018). The vision module is a reconstruction-anchored Finite Scalar Quantization (FSQ) tokenizer (Mentzer et al., 2024) over Sobel edge maps. The model is a transformer that interleaves actions with frame blocks and predicts the next token grid in parallel. The controller combines the current token grid with the transformer's temporal state, receives a short BC warm start, and is then optimized by PPO (Schulman et al., 2017) entirely in frozen-model rollouts. At deployment, the decoder is absent and the system executes one reactive pass per frame; expensive policy optimization has already happened offline.

This is a systems co-design contribution rather than a new generic world-model primitive. The compact tokenizer and parallel next-frame predictor bound inference cost; explicit action conditioning permits counterfactual rollouts; BC avoids an expensive random-policy transient; and GPU-resident state plus captured execution remove avoidable synchronization from the live path. Our contributions are:

1. an end-to-end discrete world-model controller for a non-paused game, with visual dynamics learned from captured trajectories and a BC-initialized policy optimized in imagined rollouts seeded from recorded contexts, then deployed from live screen pixels;

2. controlled live comparisons of the frozen PPO policy against BC and no-op controls across the first three official levels; and

3. an end-to-end latency profile on consumer hardware, matched action interventions, and qualitative interactive rollouts demonstrating open-ended visual level continuation together with its recurrent failure modes.

2 Related work

World models and imagination. The original Vision–Model–Controller pipeline compressed observations with a VAE, modeled latent dynamics with an MDN–RNN, and demonstrated in VizDoom that a compact controller could be optimized entirely in learned dreams and transferred to the real environment (Ha and Schmidhuber, 2018). IRIS retains this imagination-only policy-learning paradigm while replacing recurrent continuous-latent dynamics with a discrete autoencoder and autoregressive transformer, scaling it to data-efficient learning on Atari 100k (Micheli et al., 2023). Δ-IRIS reduces the sequence cost of this template by separating stochastic changes into discrete tokens from continuous context tokens (Micheli et al., 2024). DIAMOND removes discrete compression from the dynamics model and uses diffusion to preserve control-relevant visual details (Alonso et al., 2024), while DreamerV3 demonstrates broad control with categorical recurrent state-space latents (Hafner et al., 2025). Contemporaneous MIRA scales interactive world modeling to tightly coupled multiplayer dynamics, generating real-time four-player Rocket League rollouts conditioned on each player's actions (Hu et al., 2026). MineWorld likewise interleaves discrete visual and action tokens and predicts each frame's spatial tokens in parallel, while Matrix-Game 2.0 uses few-step diffusion for streaming action-conditioned video (Guo et al., 2025; He et al., 2025). These systems target interactive video generation. DashVMC instead uses a compact discrete world model to optimize a policy in imagination and closes the control loop through the original non-paused game on consumer hardware. Its transformer also uses the action-conditional contrastive auxiliary objective introduced by TWISTER (Burchi and Timofte, 2025). Earlier screenshot-based Geometry Dash control used direct DQN and imitation learning under a DLL-fabricated timestep, and highlighted that narrow jump windows can make nearly identical frames require different actions (Li and Rafferty, 2017). Our protocol leaves the game clock running and supplies the controller with temporal context through the learned world model.

Compact tokenization and policy initialization. FSQ replaces learned vector-quantization codebooks with bounded scalar levels, avoiding commitment losses and codebook collapse while exposing an integer coordinate lattice (Mentzer et al., 2024); we use the improved iFSQ bounding function (Lin et al., 2026). For policy initialization, demonstration-based RL has repeatedly shown that supervised behaviour can reduce the expensive early interaction phase (Hester et al., 2018; Baker et al., 2022). Here demonstrations only initialize the controller: the final policy is selected after PPO improvement in the learned environment.

Representation anchors and alternatives. Joint-embedding prediction can avoid spending capacity on high-magnitude input detail that is irrelevant or unpredictable from context (Van Assel et al., 2025), and LeWorldModel demonstrates stable end-to-end predictive learning from pixels with continuous latents (Maes et al., 2026). Our observation is instead a deterministic Sobel rendering with much nuisance detail already removed, making reconstruction a comparatively inexpensive anchor for a shared discrete token space. The pre-freeze joint-training diagnostic in Section 5.4 is therefore a constrained FSQ-token adaptation, not a faithful JEPA reproduction or evidence against joint embedding. Likewise, Gaussian Quant reports stronger image rate–distortion results than FSQ on large image benchmarks (Xu et al., 2025); FSQ is used here for stability, explicit lattice structure, and measured deployment cost rather than claimed tokenizer optimality.

3 DashVMC

The three modules are trained sequentially: V and M are frozen before controller fitting, and the selected controller is frozen for deployment. Figure 1 separates the imagined PPO path from the live reactive path.

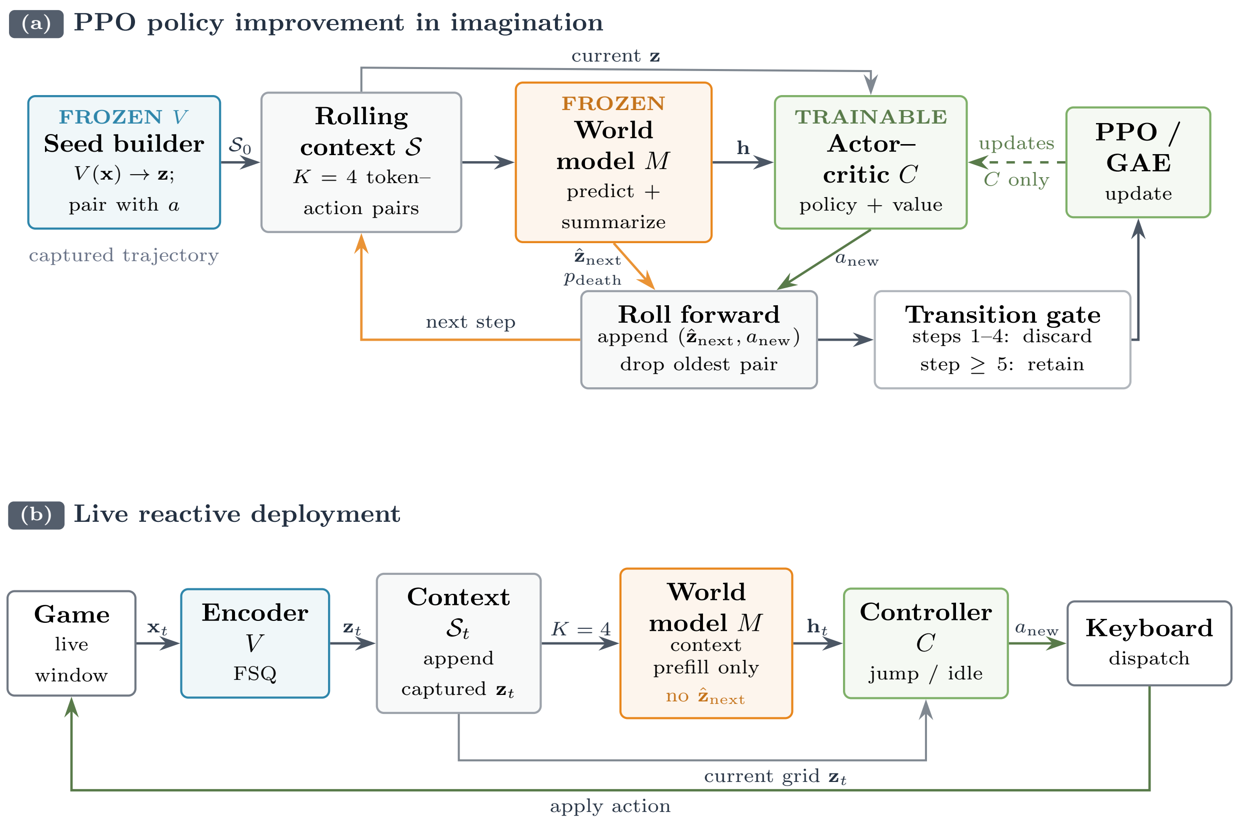

Figure 1 DashVMC policy improvement and deployment. **(a)** PPO rollouts start from a recorded $K = 4$ token–action context. The frozen world model predicts $\hat{\mathbf{z}}_{\text{next}}$ and p_{death} while returning context feature $\mathbf{h}$; the predicted grid and controller-selected action are appended to the rolling context. Four imagined burn-in steps replace the recorded seed before transitions are retained, and PPO updates only C. **(b)** During live control, all weights are frozen. V encodes captured frames and M performs context prefill only to produce $\mathbf{h}_t$. The controller receives the current grid $\mathbf{z}_t$ directly together with $\mathbf{h}_t$ and dispatches the keyboard action; neither next-grid prediction nor decoding is run.

3.1 Visual tokenization

Each RGB capture is cropped, converted to grayscale, filtered with Sobel edges, and resized to 64×64. A convolutional autoencoder maps the result through three stride-2 stages to a four-channel 8×8 latent map. Each spatial vector is independently quantized with iFSQ levels $[8, 5, 5, 5]$, using the bounding function $2\,\text{sigmoid}(1.6z) - 1$, and yields 64 token IDs from 1000 implicit codes. The tokenizer is trained on adjacent frame pairs with pixel-MSE reconstruction plus temporal slowness and latent-uniformity regularizers (Xia et al., 2025). After selection, the encoder is frozen and the decoder is retained only for visualizing dreams.

3.2 Action-conditioned dynamics

The dynamics model has 8 transformer layers, width 384, and 8 attention heads. Each frame block contains 64 visual tokens and one ALIVE/DEATH status token; action tokens are inserted between blocks. Given $K = 4$ observed frame–action pairs, a final masked block requests the next frame. The resulting sequence has $K(65+1)+65 = 329$ positions; every target-frame position receives a learned `[MASK]` embedding, so no target token is exposed during training. Attention is bidirectional within a frame and causal across frame/action blocks, so all next-frame positions are predicted in one forward pass without seeing their targets. Factored 3D rotary embeddings encode row, column, and time, with spatial and temporal bases 10 and 10,000; status and action tokens use the grid centre coordinates (Su et al., 2024). The status logits supply the rollout termination score, and the final context state $\mathbf{h}_t \in \mathbb{R}^{384}$ summarizes recent dynamics for the controller. In the reported interleaved sequence, this state is taken from the most recent action-token position. Because death frames are oversampled during training, the DEATH score from a two-class softmax over ALIVE/DEATH is used operationally rather than interpreted as a calibrated environment probability.

We add TWISTER's action-conditional contrastive predictive-coding loss with weight 0.1 (Burchi and Timofte, 2025): for each future offset $k \leq K$, an MLP predicts $\mathbf{h}_{t+k}$ from $\mathbf{h}_t$ and the intervening actions and is scored with in-batch InfoNCE. We also independently replace 5% of context tokens uniformly and 5% with adjacent FSQ codes. The token loss uses focal modulation (Lin et al., 2017) and a fixed local FSQ-coordinate target. For target code i with lattice coordinate $\mathbf{c}_i$,

$$q(j \mid i) = \begin{cases} 1 - \varepsilon, & j = i, \\ \varepsilon \dfrac{\exp(-\|\mathbf{c}_i - \mathbf{c}_j\|_2^2 / 2\sigma^2)}{\sum_{\ell \neq i} \exp(-\|\mathbf{c}_i - \mathbf{c}_\ell\|_2^2 / 2\sigma^2)}, & j \neq i, \end{cases} \tag{1}$$

with $\varepsilon = 0.1$ and $\sigma = 0.9$. Unlike uniform label smoothing (Szegedy et al., 2016), this assigns the smoothing mass locally on the FSQ lattice. Writing $H(q,p) = -\sum_j q(j \mid i) \log p_j$, focal modulation is applied once to the aggregate soft-target loss as $(1-e^{-H})^2 H$, rather than independently per class; the output projection is tied to the token embedding. Decoder perturbations motivated this target: adjacent FSQ coordinates reconstructed nearly identical patches, with visual error increasing with coordinate offset. In an archived same-architecture comparison on an earlier 256-width model, replacing uniform smoothing with SLS improved validation token accuracy (32.85% to 33.66%) and CPC loss (0.345 to 0.238) at an essentially unchanged train–validation gap. This single-run development comparison supports the local choice; a later matched IRIS/Pong diagnostic does not support a general claim (Section 6).

3.3 Controller and latent policy improvement

The $\sim$45K-parameter actor–critic embeds the current 8×8 token grid, processes it with two convolutions and max pooling, concatenates the resulting 256-dimensional feature with $\mathbf{h}_t$, and uses zero-initialized heads to predict jump probability, value, and an eight-action auxiliary output covering the current and next seven actions. The controller is first fit to expert actions by BC, then optimized with PPO in autoregressive rollouts from the frozen world model.

Each PPO iteration starts from recorded four-frame contexts and generates 512 imagined episodes of at most 45 steps. The first four imagined transitions refresh the rolling context and are excluded from PPO updates; retained transitions therefore use an entirely model-generated four-frame context. The 45-step horizon roughly covers obstacles visible in the recorded seed. Beyond it, they have usually left the crop, so the model often enters a physically valid but uninformative empty segment; later obstacles must also be generated rather than seed-observed, progressively shifting controller inputs away from encoded real frames. We stop at this boundary to learn consequences of observed obstacles without relying on invented challenges. Rollouts terminate when the predicted death score exceeds 0.5. Reward is +1 per survived step and -0.25 per jump, preventing an always-jump local optimum. We use GAE with $\gamma = 0.995$, $\lambda = 0.95$, PPO clip 0.2, and auxiliary-action loss weight 0.1.

3.4 Reactive deployment path

The training corpus is captured at a nominal 30-FPS cadence, but the optimized live stack is deployed at 60 FPS. Captured images enter pinned host memory and are transferred asynchronously; token context remains in a GPU-resident ring buffer. At each step, the transformer runs context prefill only to obtain $\mathbf{h}_t$; its masked next-grid prediction phase is not executed. The FSQ encoder uses `torch.compile`, and transformer context encoding is captured in a CUDA Graph after warm-up. The decoder is omitted, and a keyboard update is issued only when the binary action changes. The loop sleeps after action dispatch to maintain the selected cadence, but that deliberate sleep is excluded from capture-to-action latency. This is deliberately reactive rather than an inference-time planning system: online search or trajectory optimization would have to share the same 16.7-ms (60-FPS) budget with capture and dispatch.

4 Experimental setup

Data and training. The unaugmented corpus contains 4,264 base gameplay episodes and 251,963 source frames, equivalent to approximately 2.33 hours at the nominal 30-FPS capture cadence. It includes deliberate failures and 36 expert trajectories or segments. The base episodes span primarily official Levels 1–7 and multiple community-created levels. Episodes containing mechanics outside the selected model's scope, including gravity inversion, were excluded to keep the learned dynamics tractable at this scale. The expert subset comprises 33,153 frames; at the nominal 30-FPS capture cadence this is 18.42 minutes, while the recorded wall-clock capture duration is 18.95 minutes (mean capture rate $\approx$29.16 FPS). Capture metadata do not identify every segment's in-level start state or completion status, so expert segments are not treated as verified full-level clears. Episodes are split once, 90/10, by base identifier and source with seed 42; vertical shifts of ± 2 and ± 4 pixels inherit the base assignment and expand the tokenized corpus to 21,320 episodes. The frozen encoder retokenizes the corpus before dynamics training. All stages use seed 42; primary training is sequential in BF16 on one A100, while live inference is measured on an RTX 2060 SUPER. Table 1 records the optimization and selection details that materially determine the frozen system.

Table 1 Training and checkpoint selection. Tokenizer/dynamics learning rates use cosine decay after any stated warm-up.

Stage	Optimization	Selection
Tokenizer	Adam; 1,000 epochs; batch 2,048; cosine LR from 10^{-3} to 10^{-5}; slowness 0.1; uniformity 0.01; validation every 10 epochs.	Min. loss (epoch 920).
Dynamics	AdamW; 200 epochs $\times$ 500 steps; batch 512; LR $2\times10^{-3} \to 5\times10^{-5}$ after 5-epoch warm-up; weight decay 0.01; dropout 0.1; max grad. norm 1.0; death frames oversampled 5$\times$.	Max. death F1 (epoch 139).
BC	AdamW; 50 epochs; batch 512; LR 10^{-3}; weight decay 10^{-4}; positive jump weight 1.5.	Min. loss (epoch 9).
PPO	Adam; 512 rollouts/iteration; four update epochs; minibatch 512; LR 10^{-4}; entropy/critic weights 0.01/0.5; ratio clip 0.2; max grad. norm 0.5.	Max. development survival (iteration 3,250).

Checkpoint and live evaluation. PPO checkpoint selection uses survival on 512 fixed latent development contexts every ten iterations. Because training seeds may originate from the same episode collection and selection is adaptive, this is explicitly a development metric, not held-out validation. For live evaluation, all checkpoints are frozen and Auto-Retry is enabled. Auto-Retry only standardizes restarts and supplies no observation to the policy; the policy receives neither a level identifier, elapsed time or frame index, level progress, nor game memory. Live actions are deterministic: jump is selected exactly when the controller output exceeds 0.5. We retain 99 scored PPO attempts, 100 BC attempts, and 10 no-op attempts on each of the first three official levels. Two auxiliary live evaluations reuse the same Auto-Retry protocol with smaller sample sizes: 20 PPO, 20 BC, and 10 no-op scored attempts at the 30-FPS evaluation cadence. The first uses *Stereo Madness Copy* (KRUTOYARBUS), a community copy of a later Level-1 segment, as a late-stage/ship-reachable proxy because official Level 1–3 attempts die before ship. The second uses one custom community level, *Stereo INSANE Nerfed* (XXcapta1n), as a single held-out layout within the supported mechanic distribution (no gravity inversion or other out-of-scope mechanics). This level was first accessed for this evaluation after the corpus and checkpoints were frozen and is therefore absent from the training data. In addition, a cadence-transfer probe reruns the frozen PPO controller for 20 scored attempts on Stereo Madness Copy at 60 FPS on the optimized deploy-equivalent inference path (FSQ `torch.compile`, transformer CUDA graph, GPU-resident context, no per-stage CUDA synchronization). The deterministic no-op diagnostic is repeated ten times because it always reaches the first obstacle; the repeats quantify capture and action-timing jitter, with a standard deviation of 0.7 frames on every official level and 0.8 frames on the held-out custom level. Each invocation's manually resumed synchronization episode is excluded; game memory is used only to delimit alive/dead episodes, never as policy input. The primary endpoint is frames survived; when comparing 30-FPS and 60-FPS deployments, we use the recorded episode wall time from evaluation start to detected death because frame counts are not comparable across cadences. Reported intervals are 95% percentile-bootstrap confidence intervals for the policy mean using 50,000 resamples. The original selected BC checkpoint used to initialize PPO was not archived, so the live comparator uses an epoch-9 reconstruction whose validation accuracy is close, but not bit-identical, to the archived trace (79.85% versus 79.76%). Deployment measurements use Desktop Duplication via `dxcam` over a 1920×1080 region, crop $(x, y, s) = (660, 48, 1032)$, Sobel preprocessing to 64×64, and `keyboard` space dispatch on PyTorch 2.11.0+cu126.

Latency and continuation protocols. Latency is measured on 5,000 consecutive frames of the optimized deployment path, from capture through action update, with CUDA synchronization only at the end-to-end boundary. For the qualitative intervention, the world model is initialized from a fixed encoded four-frame prefix immediately before a spike. Greedy continuations differ only in the first conditioned action, jump or idle; all subsequent actions are idle. The display-only HUD is removed before re-encoding the context.

5 Results

5.1 Frozen-system and live-control results

All component metrics below are reported at their checkpoint-selection points rather than as independent maxima. The selected tokenizer reaches 3.894×10^{-4} per-pixel validation MSE (34.10 dB PSNR), obtained by dividing its per-frame reconstruction SSE of 1.595 by 64^2. Under the training-time full-vocabulary argmax metric, the transformer reaches 29.74% token accuracy and death precision/recall/F1 of 0.7337/0.8654/0.7941; low exact-token accuracy can coexist with useful local predictions in a 1,000-class grid, but it also bounds dream reliability. Archived selected-BC accuracy is 79.76% (79.85% for the reconstructed live comparator), and the selected PPO checkpoint survives 29.71 of 45 steps on the fixed latent development contexts.

Table 2 Live frame survival (mean $\pm$ population standard deviation). Attempts are repeated restarts of one frozen policy; n is per level.

Policy	n	Level 1	Level 2	Level 3
No-op	10	46.2 ± 0.7	63.4 ± 0.7	41.5 ± 0.7
BC	100	130.4 ± 91.1	119.7 ± 71.5	45.2 ± 9.0
PPO	99	$\mathbf{279.4 \pm 48.0}$	$\mathbf{262.9 \pm 121.2}$	$\mathbf{63.8 \pm 27.1}$

Table 2 shows that the selected PPO policy adds 149.0 and 143.2 mean frames over the reconstructed BC comparator on Levels 1 and 2, more than doubling survival; the PPO mean's 95% bootstrap intervals are $[270.2, 289.0]$ and $[239.4, 287.2]$. This establishes a practical policy-level difference, but not an exact before/after causal estimate against PPO's unavailable parent checkpoint. At the fixed 30-FPS cadence, PPO's three means correspond to 9.31, 8.76, and 2.13 seconds of survival. The Level-3 gain is only 18.6 frames (PPO mean 95% interval $[58.8, 69.5]$). Level 3 is harder and introduces yellow orbs requiring a second timed input while airborne, so these per-level results are not a controlled test of generalization. All three official live-evaluation levels occur in the offline corpus, but the controller receives no live-game interaction during PPO; the live ordering is therefore consistent with imagined policy optimization transferring to deployment, not evidence of held-out-level generalization. Table 3 reports the two auxiliary live evaluations under the same frozen checkpoints. On the Level-1 copy, no-op dies deterministically at 135 frames, reconstructed BC reaches 271.2 ± 107.0, and PPO reaches 525.6 ± 51.6 (PPO mean 95% interval $[503.9, 548.4]$), preserving a clear PPO >

reconstructed BC > no-op ranking on a later/ship-reachable segment that official Level 1–3 evals never enter. On the single custom held-out level, the same ordering transfers at lower absolute survival: no-op 46.1 ± 0.8, reconstructed BC 113.7 ± 69.6, and PPO 215.7 ± 75.7 (PPO mean 95% interval $[184.1, 249.9]$). We report the custom-level result as held-out transfer within the supported mechanic distribution, not as novel-mechanic or broad out-of-distribution generalization; the Level-1 copy is not counted as held-out content. The 60-FPS cadence probe on Stereo Madness Copy yields similar observed real-time survival: mean recorded episode wall time is 17.03 s at 60 FPS versus 17.79 s at 30 FPS, with a 95% bootstrap interval of $[-1.80, 0.31]$ s for the mean difference (60 FPS minus 30 FPS). Both conditions die in the same recurring 15–21 s band; frame counts roughly double under the higher cadence, as expected, so the cross-cadence comparison uses recorded time rather than frames. This supports practical 60-FPS deployment from 30-FPS training data on this layout, without claiming broad frame-rate adaptation as a separate contribution. Separate intervention runs suggest that skipping a few recurring blocking obstacles lets the policy reach later sections, but those assisted runs are not part of the controlled statistics.

Table 3 Auxiliary live frame survival (mean ± population standard deviation). Stereo Madness Copy is a late Level-1 community copy used as a ship-reachable segment proxy; Stereo INSANE Nerfed is one custom held-out community level. Sample sizes are 10 no-op / 20 BC / 20 PPO scored attempts.

Policy	n	Stereo Madness Copy	Stereo INSANE Nerfed
No-op	10	135.0 ± 0.0	46.1 ± 0.8
BC	20	271.2 ± 107.0	113.7 ± 69.6
PPO	20	$\mathbf{525.6 \pm 51.6}$	$\mathbf{215.7 \pm 75.7}$

5.2 Capture-to-action latency

In a separate 5,000-frame optimized-path profile collected against the 30-FPS reference cadence, end-to-end latency averages 16.289 ms (median 16.308 ms, p95 18.722 ms); only 2 frames exceed 33.3 ms, and the reciprocal mean is about 61 FPS of compute-equivalent throughput. More directly, the configured 60-FPS Stereo Madness Copy probe records 300 full-loop measurements with mean 14.420 ms, median 14.549 ms, p95 16.121 ms, and maximum 17.038 ms. Only 4/300 measurements exceed the 16.667 ms frame period, and even the slowest corresponds to 58.69 FPS of compute-equivalent throughput.

Table 4 Separate 5,000-frame optimized-path profile on an RTX 2060 SUPER, collected against the 33.3 ms (30-FPS) reference cadence. Component values are means in milliseconds; end-to-end is measured independently and includes dispatch overhead.

Component	ms	Component	ms
Screen capture	2.2	Crop	0.5
Grayscale	0.5	Sobel	2.5
Downscale	2.2	FSQ encoder	2.1
Transformer	4.9	Controller	0.5
Component sum	15.3	**End-to-end mean**	**16.3**
p95	18.7	Frames over 33.3 ms	2/5,000

These timings differ from diagnostic evaluator logs that move context through CPU/NumPy and synchronize after each GPU stage. Official-level and 30-FPS auxiliary survival use that diagnostic path; the 60-FPS cadence probe and deployment latency use the optimized path. Both paths apply the same preprocessing and frozen model/controller weights; the distinction is execution optimization rather than policy definition.

5.3 Action-conditioned continuations

World-model generation is not capped by the 45-step PPO horizon: each predicted grid and human-selected action can be fed back into the rolling context, so the interactive loop can continue for an arbitrary number of steps unless predicted death or an optional user limit stops it. Recorded examples can remain visually and mechanically plausible through substantial stretches, including height transitions and avatar transformations, before drift produces repeated motifs, empty segments, or impossible geometry. We therefore treat this as qualitative, open-ended level continuation rather than indefinitely coherent generation. Additional successes and failure cases are available on the project page.

MIRA emphasizes that action conditioning does not itself establish action adherence and evaluates commanded-action recoverability from generated video (Hu et al., 2026). Our matched-prefix test is a smaller qualitative intervention check rather than a population-level adherence metric. Figure 2 tests action sensitivity while holding the visual context fixed. The jump-conditioned branch clears the spike and lands; the idle-conditioned branch reaches a death score of 0.74 at $t+5$ and terminates. Decoding is used only for inspection, not controller training.

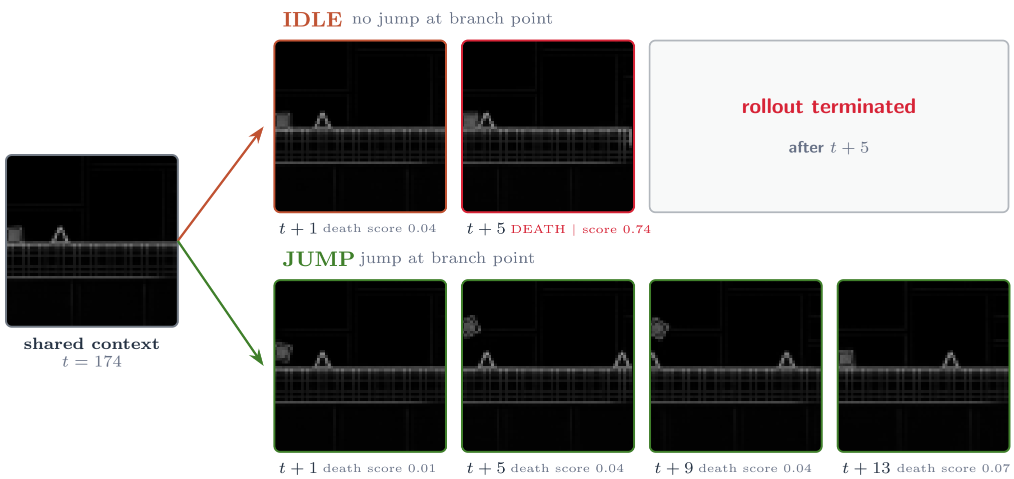

Figure 2 Greedy continuations from one reconstructed four-frame prefix; only the first future action differs. At $t+1, t+5, t+9, t+13$, the jump branch clears the spike. The idle branch terminates at $t+5$ (death score 0.74), so later frames are not generated.

5.4 Design diagnostics

Table 5 condenses engineering variants explored before freezing the system. These diagnostics span successive development stacks; except for the quantified BC timing comparison, they are local selection records rather than balanced, directly comparable statistical ablations. They explain design choices but do not establish general architectural rankings. In particular, the joint variant is an FSQ-token adaptation inspired by LeWorldModel, not a reproduction of its continuous-latent JEPA or SIGReg regularization (Maes et al., 2026).

Table 5 Pre-freeze design diagnostics from successive development stacks. "Live" denotes real-game progress in local evaluation.

Variant	Observed outcome	Decision
PPO without BC (earlier stack)	Same latent plateau, about 2,000 more iterations (≈ 6 A100 hours); BC takes about 5 minutes.	Keep BC.
Joint FSQ+transformer	Lower validation CE but worse live progress; removing the pixel anchor collapsed the encoder.	Freeze tokenizer.
AdaLN action conditioning (Peebles and Xie, 2023)	Canonical zero-gate initialization stalled; bias-1 gates plus QK-normalization (Esser et al., 2024) trained stably and reached the best development CPC. The trial co-varied other components and did not establish a live advantage over action tokens.	Use action tokens.
Transformer width 512	Better CE/contrastive metrics, no live gain, slower training.	Use width 384.
MLP on $\mathbf{h}_t$ only (later 512-width stack)	PPO was 2–5× faster with a smaller train–eval gap, but local live progress was worse; an earlier stack showed the opposite ordering.	Keep grid input.

Historical run records and code snapshots support these pre-freeze diagnostics; the exact final-system claims above use the frozen artifacts specified in the evaluation protocol.

6 Limitations

DashVMC is a single-game study with binary actions, deterministic layouts, and one training seed. The hundreds of live attempts quantify deployment variability for one frozen policy, not uncertainty across training runs, and survival time is not level completion. The auxiliary custom-level evaluation uses a single held-out community layout and smaller sample sizes than the official-level suite; it supports within-mechanic transfer rather than broad generalization. The Level-1 copy evaluates later-stage survival but does not isolate ship mode with explicit mode labels. The 60-FPS cadence comparison is a single-level, single-policy probe with $n = 20$ and also changes from the diagnostic to the optimized execution path; it is an operational deployment probe, not an isolated or broad study of frame-rate adaptation. The BC control is an epoch-9 reconstruction close to the archived training trace, not the missing original checkpoint; consequently, PPO versus BC is not an exact comparison to PPO's parent policy. Because no other world-model agent is available under the same captured-pixel, non-paused Geometry-Dash protocol, the results are a controlled within-system comparison rather than a state-of-the-art benchmark.

The Geometry-Dash evidence for Equation 1 is a single-run, same-architecture comparison against uniform smoothing, not a replicated hard-CE ablation; it supports the local choice but neither quantifies uncertainty nor isolates transfer to the final system. In a matched IRIS/Pong diagnostic with five seeds per condition, annealed SLS underperformed CE in final return (12.14 ± 10.82 versus 15.68 ± 4.96) and doubled the tail-failure rate (40% versus 20%), despite lower drawdown and return volatility and a transient advantage during epochs 250–420. We therefore treat any benefit as specific to the structured FSQ geometry and Geometry-Dash setting rather than claim a general discrete-world-model improvement. The open-ended decoded continuations are qualitative: coherence is not scored, and autoregressive drift can yield repetitions, empty stretches, or impossible states; they are neither editor-valid nor guaranteed playable.

Successive development-time dataset doublings produced visibly more coherent, higher-quality rollouts without an observed plateau, suggesting that the present model remains data-limited; these runs were not controlled end-to-end scaling experiments. Although the system is learned from a small offline corpus, we do not report a data-scaling curve or matched larger-data baseline. The results therefore demonstrate viability under a constrained data budget, not general sample efficiency.

The reported 14.4-ms 60-FPS probe and separate 16.3-ms profile cover this hardware and 64-pixel edge observations; RGB input, stochastic environments, larger action spaces, or weaker devices require new accuracy–latency measurements.

7 Conclusion

DashVMC shows that a compact discrete world model can support both imagined policy improvement and live reactive control in a timing-sensitive game. Its selected PPO controller substantially outperforms a closely matched reconstructed BC comparator on the first two official levels and preserves that ranking on a late Level-1 copy and one custom held-out community level, while the optimized capture-to-action path sustains a 60-FPS live cadence on a consumer GPU despite 30-FPS training data. This policy ordering is consistent with imagined PPO optimization transferring to live deployment, while acknowledging that the original parent BC checkpoint is unavailable. A matched-prefix intervention shows that the model's visual future responds to the tested action change, and interactive examples show that those dynamics can produce open-ended, sometimes plausible level continuations, albeit without indefinite coherence guarantees.

Acknowledgments

We thank Florian Yger for valuable discussions and careful feedback on earlier versions of this manuscript. We thank Maël Planchot for contributing gameplay data. The NVIDIA A100 GPU used for primary training was provided by MesoNet (CRIANN, Rouen) through the Representation Learning course at INSA Rouen Normandy. Geometry Dash is developed by RobTop Games.

References

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.

Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorber, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.

Junliang Guo, Yang Ye, Tianyu He, Haoyu Wu, Yushu Jiang, Tim Pearce, and Jiang Bian. MineWorld: A real-time and open-source interactive world model on Minecraft. *arXiv preprint arXiv:2504.08388*, 2025.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.

Xianglong He, Chunli Peng, Zexiang Liu, Boyang Wang, Yifan Zhang, Qi Cui, Fei Kang, Biao Jiang, Mengyin An, Yangyang Ren, Baixin Xu, Hao-Xiang Guo, Kaixiong Gong, Size Wu, Wei Li, Xuchen Song, Yang Liu, Yangguang Li, and Yahui Zhou. Matrix-Game 2.0: An open-source real-time and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.

Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.

Anthony Hu, Václav Volhejn, Adrien Ramanana Rahary, Chris Mulder, Aditya Makkar, Alyx Liao, Amélie Royer, Manu Orsini, Adam Jelley, Eloi Alonso, Florian Laurent, Fredrik Norén, James Swingos, Jan Hünermann, Kent Rollins, Lucas Hosseini, Matthieu Le Cauchois, Maxim Peter, Pim de Witte, Tim Brown, Vincent Micheli, Moritz Böhle, Gabriel de Marmiesse, Viktoriia Sharmanska, Lucia Specia, Michael Black, and Patrick Pérez. Multiplayer interactive world models with representation autoencoders. *arXiv preprint arXiv:2607.05352*, 2026.

Ted Li and Sean Rafferty. Playing Geometry Dash with convolutional neural networks. Stanford CS231N course report, 2017.

Bin Lin, Zongjian Li, Yuwei Niu, Kaixiong Gong, Yunyang Ge, Yunlong Lin, Mingzhe Zheng, JianWei Zhang, Miles Yang, Zhao Zhong, Liefeng Bo, and Li Yuan. iFSQ: Improving FSQ for image generation with 1 line of code. *arXiv preprint arXiv:2601.17124*, 2026.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.

Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning*, 2024.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *International Conference on Computer Vision*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the Inception architecture. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self supervised learning. *arXiv preprint arXiv:2505.12477*, 2025.

Zaishuo Xia, Yukuan Lu, Xinyi Li, Yifan Xu, and Yubei Chen. Clone deterministic 3d worlds with geometrically-regularized world models. *arXiv preprint arXiv:2510.26782*, 2025.

Tongda Xu, Wendi Zheng, Jiajun He, José Miguel Hernández-Lobato, Yan Wang, Ya-Qin Zhang, and Jie Tang. Training-free vector quantization via gaussian VAEs. *arXiv preprint arXiv:2512.06609*, 2025.